

Getting Acquainted with GPU Programming
Advanced computer architecture

(B-KUL-JLI1QS)

February 22, 2024



Session Date:	February 16, 2024
Partners:	Emma Dejagere
	Brecht Van de Sijpe
Class:	MElsw
Instructor:	Alex Marinsek

1 Introduction

In this lab the input array had to be swapped, so that the element on position 1 was found on position N, element 2 was found on position N-1 and so on.
We did this using CUDA.

2 Workflow

Our code consists of a swap_elements()-function and a main()-function.

In the swap_elements()function, the unique array-index is calculated. This is necessary because when a GPU contains too many threads, the GPU will split its blocks. The unique array-index is calculated by multiplying the block index and the blocks dimension and adding the thread index. After we swap the values using: output[N - idx - 1] = input[idx].

The host memory is allocated dynamically using malloc, for the device memory we use cudaMalloc. Before the GPU operation is performed, the input data will be copied from the host to the device. This happens by using cudaMemcpy(). After the operation it is copied back from the device to the host.

In the design it was chosen to use a multiple of 4 times the amount of multiprocessors on the GPU, which can be retrieved with cudaGetDeviceCount and cudaGetDeviceProperties as blocksize, and the amount of elements in the array amount of threads.

3 Analysis

We can see the advantage of using a GPU when we increase the amount of elements in the array. When we look at the graph, we can observe that when using a small amount of elements that in terms of time it is better to use the CPU. But when the amount increases the CPU-time increases aswell. For the GPU the time remains more or less the same.

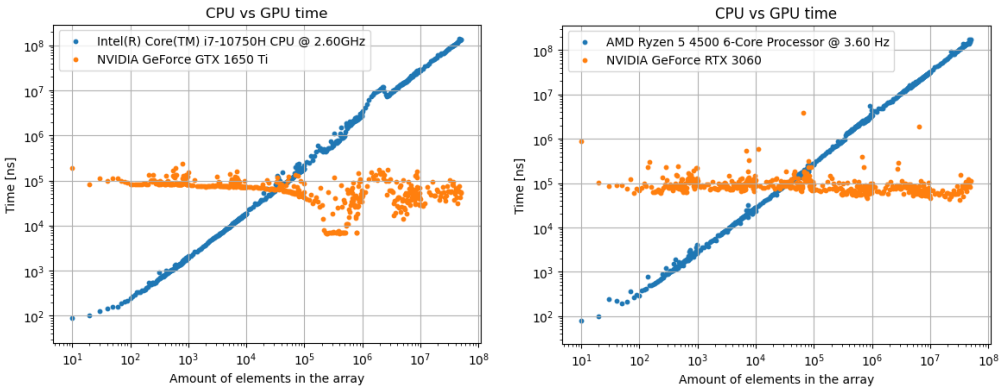


Figure 1: CPU vs GPU time.